UCLouvain
FUCaM MONS

LOUVAIN
School of Management

Louvain School of Management

Gestures Recognition

MLSM2154 : Artificial Intelligence course

Auteur : GALIZIA Matteo, MINET Bastian, SIMOENS Mathias
Professeur : SAERENS Marco
Année académique : 2025- 2026
LSM Master 1 : Ingénieur de gestion à finalité spécialisée - Business Analytics

Résumé

Hand gesture recognition is a central problem in Human–Computer Interaction (HCI), where the goal is to classify sequences of three-dimensional coordinates $\mathbf{r}(t) = [x(t), y(t), z(t)]^\top$ recorded over discrete time. This project implements and compares four classifiers on the gesture datasets introduced by Huang et al. [4], namely *domain 1* (digits 0–9) and *domain 4* (3D figures). Two baseline methods based on dynamic programming are first considered : Dynamic Time Warping (DTW) and an Edit-distance-based classifier, the latter relying on a preliminary vector-quantization step. Two state-of-the-art approaches are then investigated : the Three-Cents (3) recognizer, a 3D adaptation of the \$1 recognizer by Wobbrock et al. [12], and a Recurrent Neural Network (RNN). All methods are evaluated through leave-one-user-out (user-independent) and leave-one-gesture-sample-out (user-dependent) cross-validation procedures, and compared using hypothesis testing. This report details the theoretical framework of each method, the implementation strategy, and a rigorous statistical comparison of their accuracies.

Table of content

1 Introduction

The development of natural user interfaces has fostered a growing interest in hand gesture recognition as a means of interaction between humans and machines. Gestures, being rapidly executed freehand drawings [4], exhibit a strong inherent variability : two instances of the same gesture produced by the same user may differ in speed, scale, spatial position, and duration. The sequential nature of the signal and this variability are the main challenges any recognizer must address.

In this work we consider the problem of classifying three-dimensional gesture trajectories recorded as a sequence of spatial coordinates. Following the project guidelines, we assume constant time steps ($t = 1, 2, 3, \dots$) and ignore the precise sampling instants. Four classifiers are implemented and compared :

1. Dynamic Time Warping (DTW) as a first baseline, combined with a k-nearest-neighbors (k-NN) classifier ;
2. Edit-distance on quantized symbolic sequences, as a second baseline ;
3. The Three-Cents (3) recognizer, a 3D extension of the \$1 recognizer [12] ;
4. A Recurrent Neural Network (RNN) trained end-to-end.

Each method is evaluated on two datasets from [4] under both a *user-independent* and a *user-dependent* cross-validation protocol. The research questions underlying this project are threefold : (Q1) *how do classical template-matching baselines compare with more recent recognizers on 3D mid-air gestures ?* ; (Q2) *do user-independent settings systematically degrade accuracy, as expected ?* ; and (Q3) *is one of the investigated classifiers significantly better than the others in the user-independent setting ?*

The remainder of the report is organized as follows. Section 2 describes the datasets. Section 3 develops the theoretical framework of the four methods. Section 4 outlines the implementation and evaluation methodology. Section 5 presents and discusses the experimental results, and Section 6 concludes.

2 Data description

All the data used in this project come from the experiment of Huang et al. [4]. The complete dataset comprises four domains of ten gestures each, each gesture being performed ten times by ten different subjects, for a total of $10 \times 10 \times 10 = 1000$ samples per domain. In this work we focus on two of these domains :

— Domain 1 : hand-drawn representations of the digits $\{0, 1, \dots, 9\}$ in 3D ;

— Domain 4 : 3D geometric figures.

Each gesture is recorded as a finite sequence of 3D position vectors

$$\mathbf{r}(t) = [x(t), y(t), z(t)]^\top \in \mathbb{R}^3, \quad t = 1, 2, \dots, T_i, \quad (1)$$

where T_i is the length of the i-th sample and may vary across recordings. As stated in the project guidelines, the discrete time index is treated as uniform, even though this assumption is not strictly satisfied in practice.

3 Theoretical framework

This section introduces the four methods used in our comparative study. We first describe the two dynamic-programming baselines (DTW and Edit-distance), then the Three-Cents recognizer, and finally the Recurrent Neural Network. Throughout, we denote the test sequence by $\mathcal{X} = (\mathbf{x}_1, \dots, \mathbf{x}_N)$ and a reference (template) sequence by $\mathcal{Y} = (\mathbf{y}_1, \dots, \mathbf{y}_M)$, with $\mathbf{x}_i, \mathbf{y}_j \in \mathbb{R}^3$.

3.1 Dynamic Time Warping (DTW)

Dynamic Time Warping is a classical technique for comparing two time-dependent sequences that may differ in speed or duration [8]. The key idea is to find a non-linear alignment between the two sequences that minimizes a cumulative distance, thereby absorbing local temporal distortions.

We first define the local cost between two points $\mathbf{x}_i$ and $\mathbf{y}_j$ via the Euclidean (L_2) distance :

$$d(i, j) = \|\mathbf{x}_i - \mathbf{y}_j\|_2. \quad (2)$$

The accumulated distance $D(i, j)$ is then computed recursively using dynamic programming [9] :

$$D(i, j) = d(i, j) + \min \begin{cases} D(i-1, j-1) \\ D(i-1, j) \\ D(i, j-1) \end{cases} \quad (3)$$

with boundary condition $D(1, 1) = d(1, 1)$. The three candidate predecessors correspond respectively to a diagonal step (temporal alignment), a vertical step (the test sequence is progressing while the reference is stalling), and a horizontal step (the reference is progressing while the test is stalling).

A warping path $W = (w_1, \dots, w_L)$, with $w_k = (i_k, j_k)$, is a sequence of grid cells traversed from $(1, 1)$ to (N, M). To be physically meaningful, W must satisfy three standard constraints [9] :

— Boundary : $w_1 = (1, 1)$ and $w_L = (N, M)$;

— Monotonicity : both index sequences (i_k) and (j_k) are non-decreasing (temporal order is preserved) ;

— Continuity : transitions occur only between adjacent cells, so that no observation is skipped.

The final similarity score $D(N, M)$ is then plugged into a k-nearest-neighbors classifier : the test gesture is assigned the majority label among its k closest templates.

3.2 Edit-distance on quantized sequences

The second baseline treats gestures as strings of discrete symbols and classifies them using an edit-distance. The approach proceeds in two steps.

Vector quantization. Each 3D point $\mathbf{r}(t)$ is first assigned to one of K codewords obtained by clustering the pooled training points (via k-means). A gesture is thereby mapped onto a sequence of discrete symbols $s(t) \in \{1, \dots, K\}$, turning the continuous trajectory problem into a string-matching problem. This discretization step is necessary because edit-distance is fundamentally a string-based metric, designed to compare sequences of discrete elements rather than continuous coordinates.

Edit distance. The dissimilarity between two such symbolic sequences is measured by the Levenshtein edit distance [7], defined as the minimum number of elementary edit operations (insertion, deletion, substitution) required to transform one sequence into the other :

$$d_{\mathrm{ED}}(S_1, S_2) = \min \{\text{cost of operations to transform } S_1 \to S_2\}. \tag{4}$$

The distance is computed using a standard dynamic-programming recursion. Let $D(i, j)$ denote the edit distance between the first i symbols of S_1 and the first j symbols of S_2. Then :

$$D(i,j) = \min \begin{cases} D(i-1,j) + 1 & \text{(deletion)} \\ D(i,j-1) + 1 & \text{(insertion)} \\ D(i-1,j-1) + c(i,j) & \text{(substitution)} \end{cases} \tag{5}$$

where $c(i, j) = 0$ if $S_1[i] = S_2[j]$, and $c(i, j) = 1$ otherwise. The final value $D(|S_1|, |S_2|)$ gives the edit distance between the two complete sequences. Classification is performed with a k-nearest-neighbors rule, assigning each test gesture the label of its k nearest templates in the edit-distance metric.

3.3 The Three-Cents recognizer

The Three-Cents recognizer is a 3D specialization of the \$1 recognizer of Wobbrock, Wilson and Li [12]. The original \$1 algorithm is a widely used gold standard for 2D gesture prototyping, relying on four steps : *resample*, *rotate to a reference angle*, *scale*, and *translate to the origin*, followed by template matching. While effective, its iterative rotation search (Golden-Section Search) becomes computationally expensive and geometrically ambiguous in 3D, where no single reference angle exists.

Caputo et al. [1] propose to forgo the rotation step altogether, yielding a lightweight three-step normalization pipeline (*resample–scale–translate*) that remains robust for short mid-air trajectories.

Three-step normalization pipeline

1. **Resampling.** The raw trajectory is resampled into n equidistant points along its arc length (typically $n = 64$, as in the original \$1 paper). This makes the representation invariant to the execution speed.

2. **Scaling.** The resampled gesture is rescaled using a *uniform* scaling by total arc length. Let L denote the total arc-length of the resampled trajectory :

$$L = \sum_{i=1}^{n-1} \|\mathbf{r}'_{i+1} - \mathbf{r}'_i\|_2. \tag{6}$$

   Following [1], each point is then divided by L :

$$\mathbf{r}''_i = \frac{\mathbf{r}'_i}{L}. \tag{7}$$

   This choice differs from the bounding-box scaling used in the original \$1 algorithm, which stretches each axis independently and can distort the gesture shape. Uniform length-based scaling preserves the proportions of the trajectory, making two gestures of the same shape but different sizes comparable while preserving directional information (crucial for 3D mid-air gestures where orientation is discriminative) [1].

3. **Translation.** Finally, the gesture is translated so that its centroid lies at the origin (0, 0, 0),

$$\mathbf{r}_c = \tfrac{1}{n} \sum_{i=1}^{n} \mathbf{r}'_i, \qquad \mathbf{r}_i^{\text{final}} = \mathbf{r}'_i - \mathbf{r}_c, \tag{8}$$

   ensuring position invariance.

Classification

After normalization, the candidate gesture $C = (\mathbf{c}_1, \dots, \mathbf{c}_n)$ is compared to each template $T = (\mathbf{t}_1, \dots, \mathbf{t}_n)$ by the average Euclidean point-to-point distance :

$$D(C,T) = \frac{1}{n} \sum_{i=1}^{n} \|\mathbf{c}_i - \mathbf{t}_i\|_2. \tag{9}$$

A 1-Nearest-Neighbor rule is then applied : the candidate is assigned the label of the template achieving the minimum distance. By omitting the rotation step and relying on a fixed point-to-point correspondence, the Three-Cents recognizer provides a "cheap" alternative to alignment-based methods such as DTW, at the cost of sensitivity to global rotations.

3.4 Bidirectional LSTM

Unlike the previous methods, which are distance-based, the last approach relies on a recurrent neural network that learns its representation directly from the data. Recurrent Neural Networks (RNNs) process a sequence $\mathcal{X} = (\mathbf{x}_1, \dots, \mathbf{x}_T)$ by maintaining a hidden state

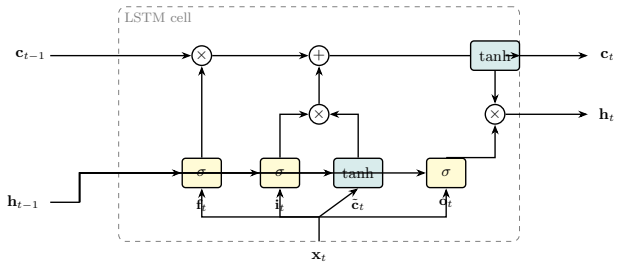

Figure 1 — *Internal data flow of an LSTM cell. Three sigmoid gates (f_t, i_t, o_t) and one* tanh *candidate ($\tilde{c}_t$) jointly update the cell state $\mathbf{c}_t$ and produce the hidden state $\mathbf{h}_t$. The horizontal line carrying $\mathbf{c}_t$ acts as a gradient highway, enabling learning over long sequences.*

that propagates information forward in time. Standard RNNs trained by back-propagation through time suffer however from the *vanishing/exploding gradient problem* : the influence of an early time step on the loss decays (or blows up) exponentially with the temporal distance, making it practically impossible to learn long-range dependencies [3, 2]. Since a gesture trajectory typically spans several dozens of frames whose global shape is the primary discriminating cue, this is a critical limitation.

LSTM cell

The Long Short-Term Memory (LSTM) cell of Hochreiter and Schmidhuber [3] addresses this issue through an internal *cell state* $\mathbf{c}_t$, regulated by three multiplicative gates. At each time step t, given the input $\mathbf{x}_t$ and the previous hidden state $\mathbf{h}_{t-1}$:

$$\mathbf{f}_t = \sigma(\mathbf{W}_f \mathbf{x}_t + \mathbf{U}_f \mathbf{h}_{t-1} + \mathbf{b}_f), \tag{10}$$

$$\mathbf{i}_t = \sigma(\mathbf{W}_i \mathbf{x}_t + \mathbf{U}_i \mathbf{h}_{t-1} + \mathbf{b}_i), \tag{11}$$

$$\mathbf{o}_t = \sigma(\mathbf{W}_o \mathbf{x}_t + \mathbf{U}_o \mathbf{h}_{t-1} + \mathbf{b}_o), \tag{12}$$

$$\tilde{\mathbf{c}}_t = \tanh(\mathbf{W}_c \mathbf{x}_t + \mathbf{U}_c \mathbf{h}_{t-1} + \mathbf{b}_c), \tag{13}$$

$$\mathbf{c}_t = \mathbf{f}_t \odot \mathbf{c}_{t-1} + \mathbf{i}_t \odot \tilde{\mathbf{c}}_t, \tag{14}$$

$$\mathbf{h}_t = \mathbf{o}_t \odot \tanh(\mathbf{c}_t), \tag{15}$$

where σ is the logistic sigmoid, $\odot$ the element-wise product, and $\{\mathbf{W}_\bullet, \mathbf{U}_\bullet, \mathbf{b}_\bullet\}$ are the learned parameters. The *forget gate* $\mathbf{f}_t$ controls how much of the previous cell state is preserved, the *input gate* $\mathbf{i}_t$ how much of the candidate $\tilde{\mathbf{c}}_t$ is written to the new state, and the *output gate* $\mathbf{o}_t$ how much of the cell state is exposed as output. The additive update of $\mathbf{c}_t$ creates a *constant error carrousel* through which gradients can flow unattenuated across many time steps, which is the key mechanism that overcomes the vanishing gradient problem [3]. Figure 1 summarizes the cell's internal data flow.

Bidirectional extension

A unidirectional LSTM reads the sequence from left to right and classifies the gesture using only the causal context up to the current frame. Since gesture recognition is an offline task — the full trajectory is available before any decision is made — this is unnecessarily restrictive. The Bidirectional RNN of Schuster and Paliwal [10] runs two independent recurrent layers in opposite directions and concatenates their hidden states :

$$\overrightarrow{\mathbf{h}}_t = \mathrm{LSTM}_{\rightarrow}(\mathbf{x}_1, \dots, \mathbf{x}_t), \quad \overleftarrow{\mathbf{h}}_t = \mathrm{LSTM}_{\leftarrow}(\mathbf{x}_T, \dots, \mathbf{x}_t), \tag{16}$$

$$\mathbf{H}_t = \left[\overrightarrow{\mathbf{h}}_t \,;\, \overleftarrow{\mathbf{h}}_t \right] \in \mathbb{R}^{2h}. \tag{17}$$

Each frame is thus encoded with access to the *entire* temporal context. This is particularly useful when the beginning of a digit is ambiguous before its closing stroke is observed (e.g., a 0 versus a 6, or a 3 versus an 8). In our implementation each direction has h hidden units (hyperparameter `n_units`), and only the final concatenated state $\mathbf{H}_T$ is forwarded to the classifier head (`return_sequences=False`).

Classifier head

The state $\mathbf{H}_T$ is passed through a small feed-forward classifier illustrated in Figure 2. Two regularization mechanisms are applied first. *Batch normalization* [5] rescales each feature of $\mathbf{H}_T$ to zero mean and unit variance over the mini-batch $\mathcal{B}$, before applying a learned affine transformation :

$$\hat{y}_k = \gamma_k \frac{y_k - \mu_{\mathcal{B},k}}{\sqrt{\sigma^2_{\mathcal{B},k} + \varepsilon}} + \beta_k. \tag{18}$$

This reduces the *internal covariate shift* caused by changing upstream weights during training, stabilizes optimization, and allows slightly higher learning rates. *Dropout* [11] is then applied with rate $p = 0.30$: during training only, each activation is independently set to zero with probability p, forcing the network to learn redundant representations and acting as an implicit ensemble of thinned sub-networks. The resulting vector is mapped through a dense layer of 32 ReLU units, then to a softmax over the $K = 10$ classes producing a valid probability distribution $p_k = e^{z_k} / \sum_j e^{z_j}$.

Training

The network is trained by minimizing the categorical cross-entropy

$$\mathcal{L} = -\sum_{k=1}^{K} y_k \log p_k, \tag{19}$$

where $\mathbf{y}$ is the one-hot encoded label. Optimization uses Adam [6], an adaptive first-order method that maintains exponentially decaying estimates of the first ($\mathbf{m}_t$) and second ($\mathbf{v}_t$) moments of the gradient and adjusts the per-parameter step size accordingly. The default hyperparameters ($\alpha = 10^{-3}$, $\beta_1 = 0.9$, $\beta_2 = 0.999$) are used. As deep models overfit easily on the $\sim$1 000-sample dataset, a 10 % validation split is held out from each training fold to monitor validation loss and *early-stop* training when no improvement is observed for 5 consecutive epochs (best weights restored). Consistent with the other methods, the model is re-initialized from scratch at every cross-validation fold so that no information leaks between folds.

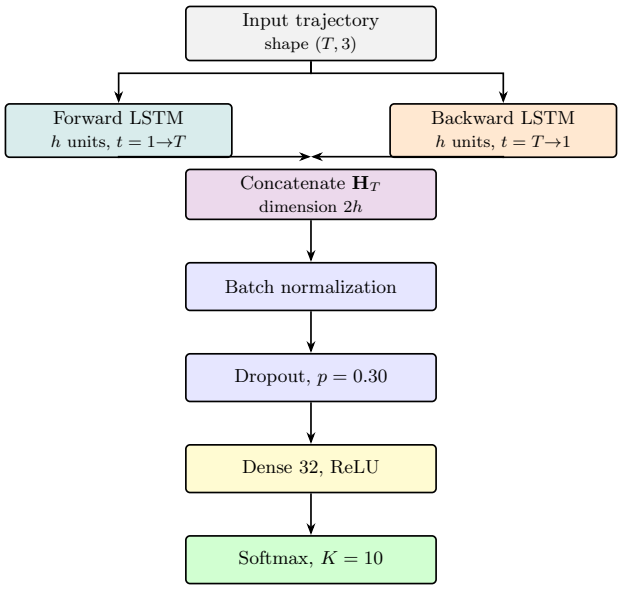

Figure 2 – *Architecture of the BiLSTM classifier. The trajectory (resampled to T points) is processed by two LSTM layers reading in opposite directions ; their final hidden states are concatenated into $\mathbf{H}_T \in \mathbb{R}^{2h}$ and passed through a regularized classifier head producing a softmax over the $K = 10$ gesture classes.*

4 Implementation and methodology

This section describes the data preprocessing, cross-validation protocols, hyperparameter selection, and experimental setup used to evaluate the four gesture recognition methods.

4.1 Data preprocessing

All gestures were loaded from the raw data files and preprocessed in a consistent manner across methods. The preprocessing pipeline consisted of three main steps :

Normalization. Each gesture trajectory was normalized using standardization (z-score normalization) computed exclusively on the training set to prevent data leakage. For each fold, the mean $\boldsymbol{\mu}$ and standard deviation $\boldsymbol{\sigma}$ were calculated from all training gesture points, and then applied uniformly to both training and test sets :

$$\mathbf{r}_i^{\text{norm}} = \frac{\mathbf{r}_i - \boldsymbol{\mu}}{\boldsymbol{\sigma}}. \tag{20}$$

This removes scale and location artifacts, ensuring that all methods operate on gesture trajectories with zero mean and unit variance.

Dimensionality reduction (optional). To assess the effect of dimensionality reduction, we optionally applied Principal Component Analysis (PCA) to the training set before classification. PCA was fit only on the training data and the resulting transformation was applied to both training and test sets. We tested PCA with $n_{\text{components}} \in \{2, 3\}$ as well as the baseline (no PCA, keeping all 3 dimensions). For gestures with variable-length trajectories, PCA was applied pointwise, preserving the temporal structure.

Variable-length handling. Gesture trajectories in both domains exhibit variable length (number of points). For methods requiring fixed-length input (Three-Cents recognizer), resampling into a fixed number of equidistant points was performed as part of the method-specific preprocessing (described below). DTW and Edit-distance handle variable-length sequences natively without modification.

4.2 Vector quantization for edit-distance

The Edit-distance baseline requires discrete symbolic sequences rather than continuous coordinates. We converted each trajectory into a sequence of cluster labels using k-means clustering, following the project guidelines.

Clustering procedure. For each fold, k-means clustering was fit on all training gesture points (pooled across all gestures and users) with a fixed random seed (42) for reproducibility. We tested cluster counts $K \in \{15, 20, 25, 30, 35, 40, 45, 50, 55, 60\}$ to identify the optimal discretization granularity. After fitting, each point in both training and test sets was assigned to the nearest cluster centroid, transforming each gesture into a string of cluster labels (e.g., $A \to A \to B \to C \to \dots$).

Compression. We also tested an optional compression step that removed consecutive duplicate symbols (e.g., $A \to A \to B$ becomes $A \to B$), reducing edit-distance sensitivity to repetitive gestures. This was tested as a binary option : compression enabled or disabled.

4.3 Cross-validation and hyperparameter selection

All methods were evaluated using two complementary cross-validation protocols to assess both user-independence and generalization within users.

User-independent evaluation. We performed leave-one-user-out cross-validation : for each of the 10 subjects in turn, we held out all gestures from that subject as the test set and trained on all gestures from the remaining 9 subjects. This was repeated 10 times (once per subject), yielding 10 accuracy measurements per method/domain/hyperparameter combination.

User-dependent evaluation. We performed leave-one-gesture-sample-out cross-validation : for each user and each gesture type, we iteratively held out one sample (repetition) as the test set, reserving the other 9

repetitions from that user for training. This was repeated 10 times (once per repetition index), again yielding 10 accuracy measurements per configuration.

Inner validation split for hyperparameter selection. To avoid any information leakage from the outer test fold into the hyperparameter search, we introduced a nested validation step. Within each outer fold, the training portion is further split randomly into an *inner training* set (80 %) and an *inner validation* set (20 %), using a fixed random seed (42) for reproducibility :

$$\mathcal{D}_{\text{train}}^{\text{outer}} \xrightarrow{80/20} \mathcal{D}_{\text{inner-train}} \cup \mathcal{D}_{\text{inner-val}}. \tag{21}$$

All preprocessing statistics (mean, standard deviation, k-means centroids, PCA components) and the method parameters are fitted *exclusively* on $\mathcal{D}_{\text{inner-train}}$. Candidate hyperparameter configurations are then scored on $\mathcal{D}_{\text{inner-val}}$. The outer test fold is thus *never* seen during hyperparameter selection, ensuring an unbiased performance estimate. Once the best configuration is identified, the full outer training set $\mathcal{D}_{\text{train}}^{\text{outer}}$ is used for the final model fit before evaluating on the held-out test fold.

Hyperparameter grid search. Within each outer fold, we performed a grid search over the inner validation set, covering hyperparameters specific to each method :

- **Edit-distance :** $K \in \{15, 20, 25, 30, 35, 40, 45, 50, 55, 60\}$ (cluster count), compression $\in$ {True, False}, and $k \in \{1, 3, 5, 7\}$ for k-NN.
- **DTW :** $k \in \{1, 3, 5, 7\}$ for k-NN ; no method-specific hyperparameters.
- **Three-Cents :** resampling to $n_{\text{points}} \in \{16, 32, 64\}$ equidistant points.
- **PCA :** $n_{\text{components}} \in \{\text{no PCA}, 2, 3\}$ applied to all methods.

For each outer fold, we computed inner-validation accuracy for all hyperparameter combinations. The best configuration per (method, domain, cv_mode) tuple was selected by maximizing mean inner-validation accuracy across folds, and then applied to the corresponding outer test fold.

4.4 Classification and distance metrics

***k*-Nearest neighbors (*k*-NN).** DTW and Edit-distance both use k-NN for classification. For each test gesture, we computed distances to all training gestures, sorted the results, and assigned the test gesture the majority-vote label among its k nearest neighbors. We tested $k \in \{1, 3, 5, 7\}$ to balance between local (small k) and global (large k) neighborhood effects.

Distance computation. For DTW, we used the accumulated dynamic programming distance $D(N, M)$ as the distance metric. For Edit-distance, we used the Levenshtein edit distance computed via dynamic programming. For Three-Cents, we used the average point-to-point Euclidean distance.

Three-Cents 1-NN. The Three-Cents recognizer uses 1-nearest-neighbor by design ; the k parameter was fixed at 1 for this method.

4.5 Implementation details and reproducibility

All methods were implemented in Python 3 using NumPy for numerical computation. DTW and Edit-distance distance computations were implemented from scratch (without external gesture recognition libraries) as required by the project guidelines. PCA and k-means clustering were sourced from scikit-learn.

Random seeds. The k-means clustering algorithm was initialized with random seed 42, and 10 different centroid initializations were tested per fold (n_init=10) to ensure robust cluster center identification.

Data leakage prevention. Critical to the experimental integrity : normalization statistics (mean, std) and clustering centroids were learned *only* from training data and applied to test data, preventing information leakage from test to train.

Metrics. Classification accuracy was computed as the proportion of correct predictions :

$$\text{Accuracy} = \frac{\text{Number of correct predictions}}{\text{Total number of test samples}}. \tag{22}$$

For each configuration, we report mean accuracy and standard deviation across the 10 cross-validation folds.

5 Results and discussion

6 Conclusion

Références

[1] Fabio M. Caputo, Pietro Prebianca, Alessandro Carcangiu, Lucio D. Spano, and Andrea Giachetti. Comparing 3d trajectories for simple mid-air gesture recognition. *Computers & Graphics*, 73 :17–25, 2018.

[2] Ian Goodfellow, Yoshua Bengio, and Aaron Courville. *Deep Learning*. MIT Press, 2016. Available at http://www.deeplearningbook.org.

[3] Sepp Hochreiter and Jürgen Schmidhuber. Long short-term memory. *Neural Computation*, 9(8) :1735–1780, 1997.

[4] Jie Huang, Shubham Jaiswal, and Rahul Rai. Gesture-based system for next generation natural and intuitive interfaces. *Artificial Intelligence for Engineering Design, Analysis and Manufacturing*, 33(1) :54–68, 2019.

[5] Sergey Ioffe and Christian Szegedy. Batch normalization : Accelerating deep network training by reducing internal covariate shift. In *Proceedings of the 32nd International Conference on Machine Learning (ICML)*, pages 448–456, 2015.

[6] Diederik P. Kingma and Jimmy Ba. Adam : A method for stochastic optimization. In *Proceedings of the 3rd International Conference on Learning Representations (ICLR)*, 2015.

[7] Vladimir I. Levenshtein. Binary codes capable of correcting deletions, insertions, and reversals. *Soviet Physics—Doklady*, 10(8) :707–710, 1966.

[8] Cory S. Myers and Lawrence R. Rabiner. A comparative study of several dynamic time-warping algorithms for connected-word recognition. *The Bell System Technical Journal*, 60(7) :1389–1409, 1981.

[9] Lawrence Rabiner and Biing-Hwang Juang. *Fundamentals of Speech Recognition*. Prentice-Hall, Englewood Cliffs, NJ, 1993.

[10] Mike Schuster and Kuldip K. Paliwal. Bidirectional recurrent neural networks. *IEEE Transactions on Signal Processing*, 45(11) :2673–2681, 1997.

[11] Nitish Srivastava, Geoffrey Hinton, Alex Krizhevsky, Ilya Sutskever, and Ruslan Salakhutdinov. Dropout : A simple way to prevent neural networks from overfitting. *Journal of Machine Learning Research*, 15(1) :1929–1958, 2014.

[12] Jacob O. Wobbrock, Andrew D. Wilson, and Yang Li. Gestures without libraries, toolkits or training : a $1 recognizer for user interface prototypes. In *Proceedings of the 20th Annual ACM Symposium on User Interface Software and Technology (UIST)*, pages 159–168. ACM, 2007.

UNIVERSITÉ CATHOLIQUE DE LOUVAIN
Louvain School of Management
Chaussée de Binche 151, 7000 Mons Belgique | www.uclouvain.be/lsm